

# Several documents on one page: sub-page regions

---

August 17, 2026

# CONTENTS

Several documents on one page: sub-page regions .....

0. The gap, measured .....

1. The constraint that decides the whole design .....

2. The architecture: the same inversion, one level down .....

2.1 Gutter detection .....

2.2 The false-positive that will kill this if it is not handled .....

2.3 The whole-page null hypothesis .....

3. Where it fits .....

4. Phases .....

5. What would make me abandon it .....

6. Open decisions .....

7. What this does not change .....

# Several documents on one page: sub-page regions

**Status:** proposed, for review. Nothing implemented. **Date:** 2026-08-17 **Builds on:** `2026-08-16-segmentation-design.md` (page-level), which this extends rather than replaces

---

## 0. The gap, measured

Segmentation is page-level throughout: a `Boundary` is a page number and a `Segment` is a page range. Two documents photocopied side by side onto one sheet cannot be separated, because the smallest unit the code has is a page.

Measured on a page carrying a W-9 and a bank statement side by side:

```
pages in file      : 1
boundaries proposed : []
segments returned  : 1
[1-1] us_w9 conf=0.766 abstained=False
```

**One confident answer, and the second document is never mentioned.** Confident rather than abstaining, which is the worse of the two failures — a reviewer has no signal that anything was missed.

Real shapes this covers: two IDs on one photocopied sheet, front and back of a card on one page, a passport bio page beside a visa page, a phone photo of several documents on a desk.

---

## 1. The constraint that decides the whole design

Sub-page work needs **geometry**, and geometry is not available everywhere. Measured:

| Reading path                                      | Blocks carrying a <code>bbox</code>                                |
|---------------------------------------------------|--------------------------------------------------------------------|
| PDF text layer (local)                            | 0 / 731                                                            |
| local OCR ( <code>rapidocr</code> )               | none — <code>ocr_pages_to_builder</code> sets no <code>bbox</code> |
| <code>azure_read</code> v3.2                      | lines only, no roles                                               |
| <code>azure_layout</code> (Document Intelligence) | 6 / 6, real coordinates                                            |

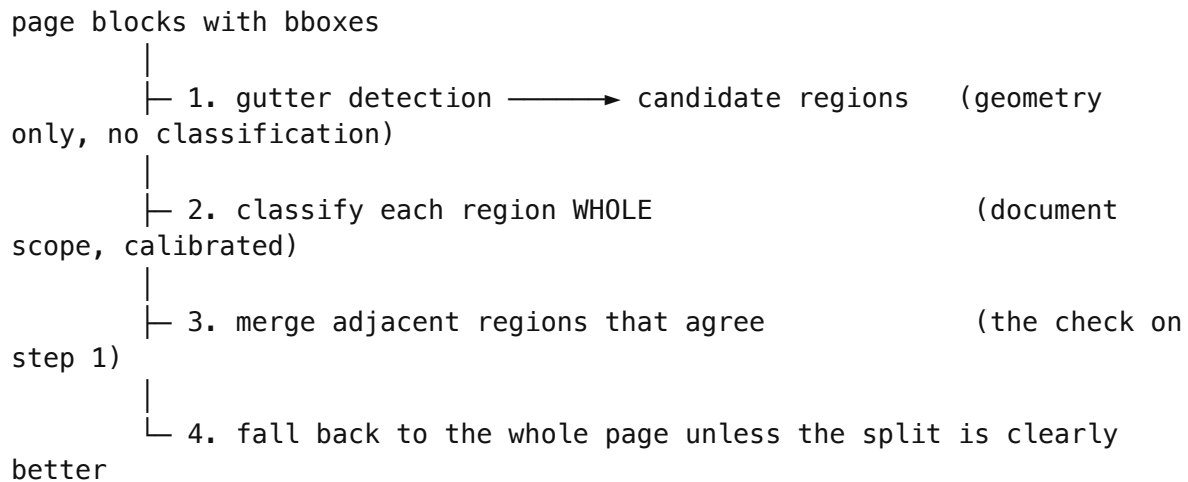
`TextBlock.bbox` is a `Quad` — 8 floats, four `(x, y)` points clockwise — and only the Azure adapters populate it ( `dce/adapters.py:222,244` , from DI `polygon` ). The local PDF parser never passes one, though `LayoutBuilder.block` accepts it.

**This is a constraint, not a problem.** A page holding two *photographed* documents is an image; an image has no text layer; an image always goes to a recogniser. So the case that needs sub-page splitting is exactly the case that already has the geometry to do it.

**Therefore: sub-page regions are an `azure_layout` -only capability, and the design should say so out loud rather than degrade quietly.** On any other path the answer stays what it is today — one segment per page — and `/readyz` should report that regions are unavailable, for the same reason the OCR posture is reported: a capability that silently is not there is worse than one that says so.

## 2. The architecture: the same inversion, one level down

Page segmentation earned its precision by refusing to classify pages. The same discipline applies here, for the same reason and with more force — a page is at least a natural unit, while a region is one we invent.



**Step 4 is new and is the safety net.** Page segmentation could afford to emit its spans because a page boundary is a real thing. A region boundary is a hypothesis, so the whole-page answer stays the null hypothesis and a region split has to *beat* it.

## 2.1 GUTTER DETECTION

Project block bounding boxes onto the X and Y axes and look for bands of page with no ink.

- A **vertical gutter** wider than ~8% of page width with blocks on both sides → candidate vertical split.
- A **horizontal gutter** taller than ~8% of page height → candidate horizontal split.
- Recurse at most twice, so a page can yield at most four regions. A photocopied sheet holds two or three documents, not nine, and an unbounded recursion is how a form becomes confetti.

Costs nothing but arithmetic over the bboxes already in hand.

## 2.2 THE FALSE-POSITIVE THAT WILL KILL THIS IF IT IS NOT HANDLED

**Multi-column layouts.** A W-9 has columns. A bank statement has columns. A naive gutter detector splits a two-column form into two “documents” — and that failure is far more common than the case the feature exists for, because almost every form has columns and almost no page has two documents on it.

Three defences, in order of strength:

1. **Classify each region whole, then merge regions that agree.** Two columns of one W-9 each classify as `us_w9` (or abstain), so they merge and the split disappears. Two genuinely different documents classify differently and survive. This is the same rule that took page-level false splits from 19.3% to 6.0%, and it is doing more work here.
2. **Require both regions to identify.** If either abstains, do not split. A column of a form read alone will usually abstain — it carries a fraction of the document's evidence — so this catches what merge does not.
3. **Require a real gutter, not a column gap.** A column gap is a few percent of page width; a gap between two photographed cards is much larger. The 8% threshold is a starting point and must be measured, not assumed.

## 2.3 THE WHOLE-PAGE NULL HYPOTHESIS

Emit regions **only** when all hold:

- two or more regions each classify to a **known** doctype (no abstentions), and
- those doctypes **differ**, and
- each region's confidence is at least as high as the whole page's

Otherwise return the single whole-page segment exactly as today. This makes the feature strictly additive: it can only ever turn one answer into several *correct* ones, never turn one correct answer into two thinner ones.

## 3. Where it fits

`DocumentSegment` gains an optional region, so the response shape barely moves:

```
{
  "start_page": 1, "end_page": 1,
  "region": { "bbox": [x0,y0, x1,y1, x2,y2, x3,y3], "index": 0, "of": 2 },
  "classification": { ... }
}
```

`region` is `null` for every segment today and for every non-DI path, so existing callers and the console are unaffected until a page actually splits.

**Extraction needs a region-scoped view**, the sub-page analogue of

`page_range_view`: filter blocks by bbox containment rather than page number. Same deep-copy, same `raw` preservation, same absolute coordinates.

## 4. Phases

| # | Work                                                                                                                                 | Ships                      |
|---|--------------------------------------------------------------------------------------------------------------------------------------|----------------------------|
| 0 | Region fixtures: real corpus documents composited side by side and stacked, plus <b>multi-column single documents as the control</b> | the measurement instrument |
| 1 | <code>region_view(view, quad)</code> — bbox-containment slicer, mirroring <code>page_range_view</code>                               | the primitive              |
| 2 | Gutter detection on block bboxes, no classification                                                                                  | candidate regions          |
| 3 | Classify-whole + merge + the whole-page null hypothesis                                                                              | the safety net             |
| 4 | <code>region</code> on <code>DocumentSegment</code> ; <code>/readyz</code> reports region availability per provider                  | the API                    |
| 5 | Console: region ranges in the verdict, and the bbox drawn if a preview exists                                                        | visibility                 |
| 6 | <b>Measure</b> : false-split rate on single multi-column documents; recall on composited pages                                       | the decision               |

**Phase 0 before anything else, and the controls matter more than the positives.**

The corpus must contain more multi-column single documents than composited pages, because the failure this design most needs to avoid is splitting a form, not missing a composite.

## 5. What would make me abandon it

Stated in advance so the decision is not made under sunk cost:

- **False-split rate above ~2% on multi-column single documents.** Page-level sits at 3.6% and that is on genuinely ambiguous long filings. Splitting an ordinary two-column form is a worse error and should clear a higher bar.
  - **The merge rule not saving it.** If two columns of one form classify as two *different* doctypes with confidence, the whole safety net fails and the approach needs rethinking.
  - **Real intake not containing the shape.** If the documents Nilendu gathers show composites are rare, this is the wrong thing to build. That question is open and is asked in the corpus request.
- 

## 6. Open decisions

1. **Is the shape real, and how common?** Needs 3–5 real examples. Everything else is contingent on this.
  2. **Front-and-back of one card on one page** — is that one document or two? It is one *instrument* photographed twice, and the honest answer is probably one segment whose evidence comes from both regions, not two segments. Different from two distinct documents and worth deciding before building.
  3. **Rotated or skewed regions.** A phone photo of documents on a desk has them at angles. `PageInfo.angle` exists but is 0.0 on every non-Azure path; DI supplies real values. Do we handle rotation in v1, or require flat-scanned input and say so?
  4. **Cost.** Each region is a classification. Four regions is four classifications of a page that today costs one — cheap, since classification is in-process and free, but worth stating.
- 

## 7. What this does not change

Page-level segmentation, the four accept gates, the egress invariant, and the abstention posture all stay exactly as they are. Regions are a refinement **inside** a page, applied after page segmentation has run, on the one reading path that carries the geometry to

support them.

If regions never ship, nothing about the current behaviour is worse than it is today.